

Understand Git & Version Management

Git is an open-source distributed version control system designed to track changes in source code, enable collaboration, and maintain a complete history of a project. Unlike traditional VCS tools that store changes as file-based deltas, Git stores snapshots of the entire project at each commit, making it efficient and reliable.

Core Git mechanics —every workflow starts with the same three-zone model:

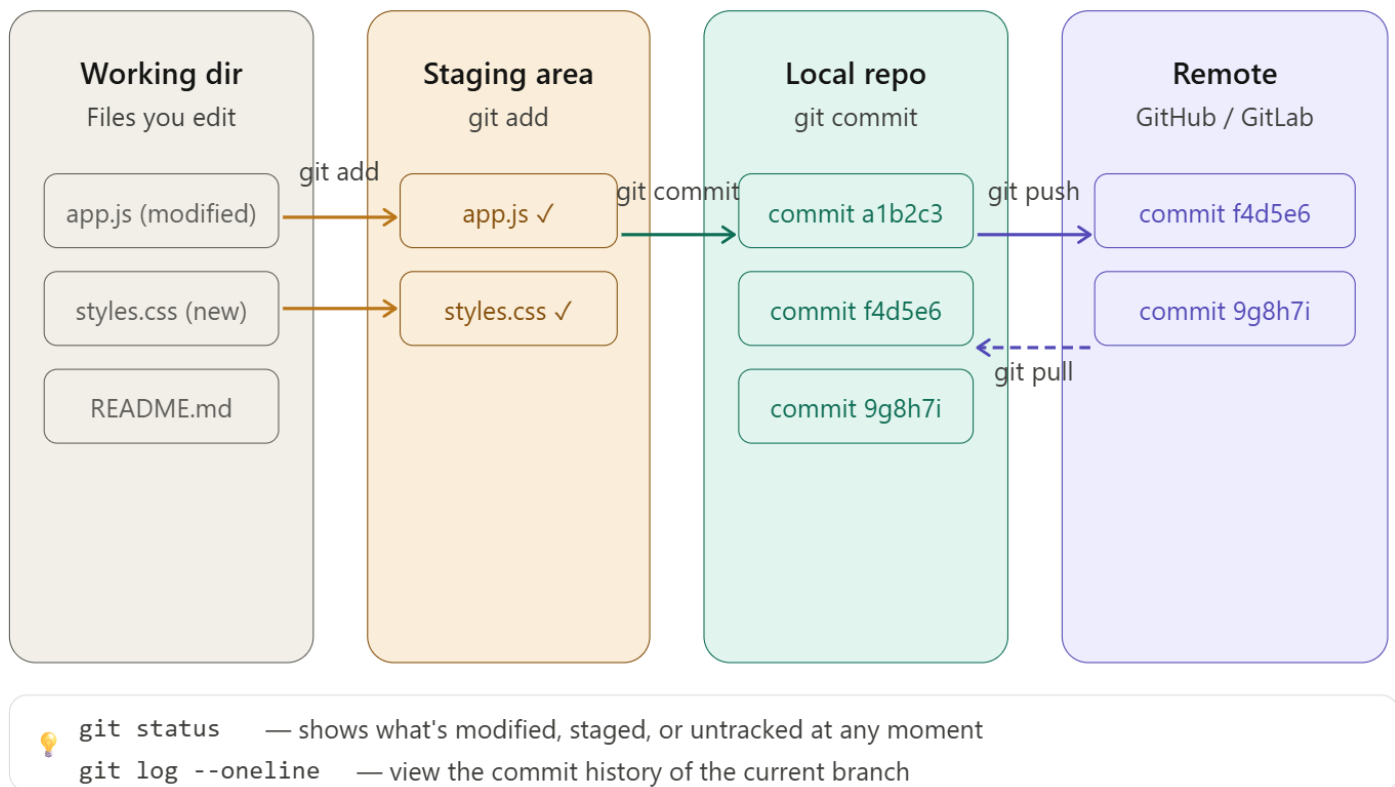
Working directory → Staging area → Local repository → Remote Repo.

The Working Directory where we actively edit files locally. This is our Playground for Dev's.

The Staging area is a temporary holding spot for change before committing.

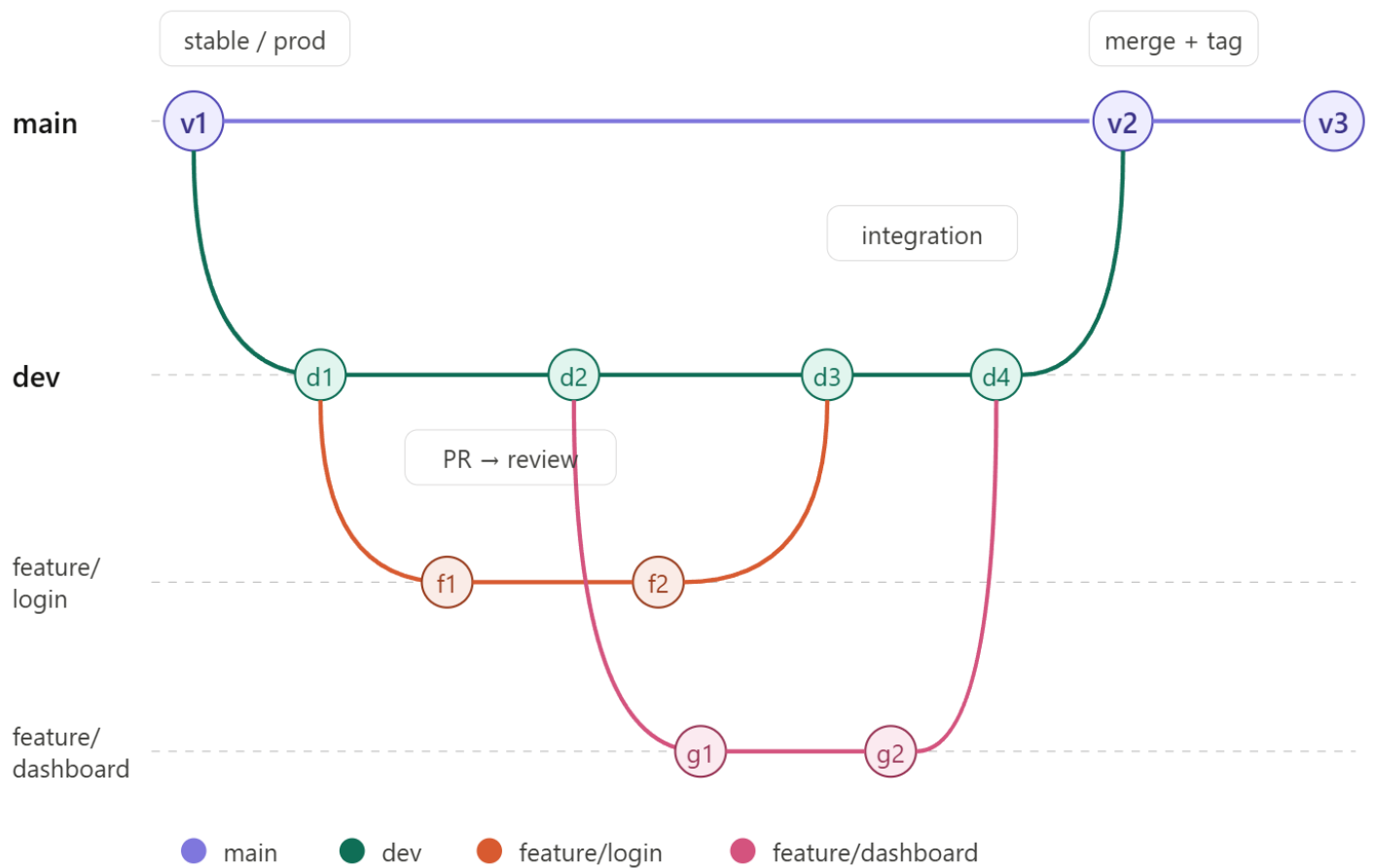
Local Repository this is where we store committed changes locally.

Remote Repository servers like github for sharing and backing up code.



Above Figure is using purpose only to represent the theory.

Branching strategies — this is where Git really shines for teams. Branches are cheap, so we can experiment without ever touching stable code.



The pattern above is **Git Flow**

- **main** — always deployable, only receives tagged releases.
- **dev** — the integration branch, features land here first for testing.
- **feature/*** — short-lived branches per task, merged back via Pull Request after code review.

What Git solves in collaborative environments:

Time travel

git revert / git reset

Broke prod? Roll back to any past commit in seconds. No undo limit.

Parallel work

branches + merge

10 devs, 10 features — nobody blocks anyone. Git merges it all.

Blame (the good kind)

git blame / git log

See who changed every line, and why. Commit msg = context.

Safe experiments

branches are free

Try a risky refactor on a branch. Abandon if it fails. Zero cost.

Distributed backup

every clone = full copy

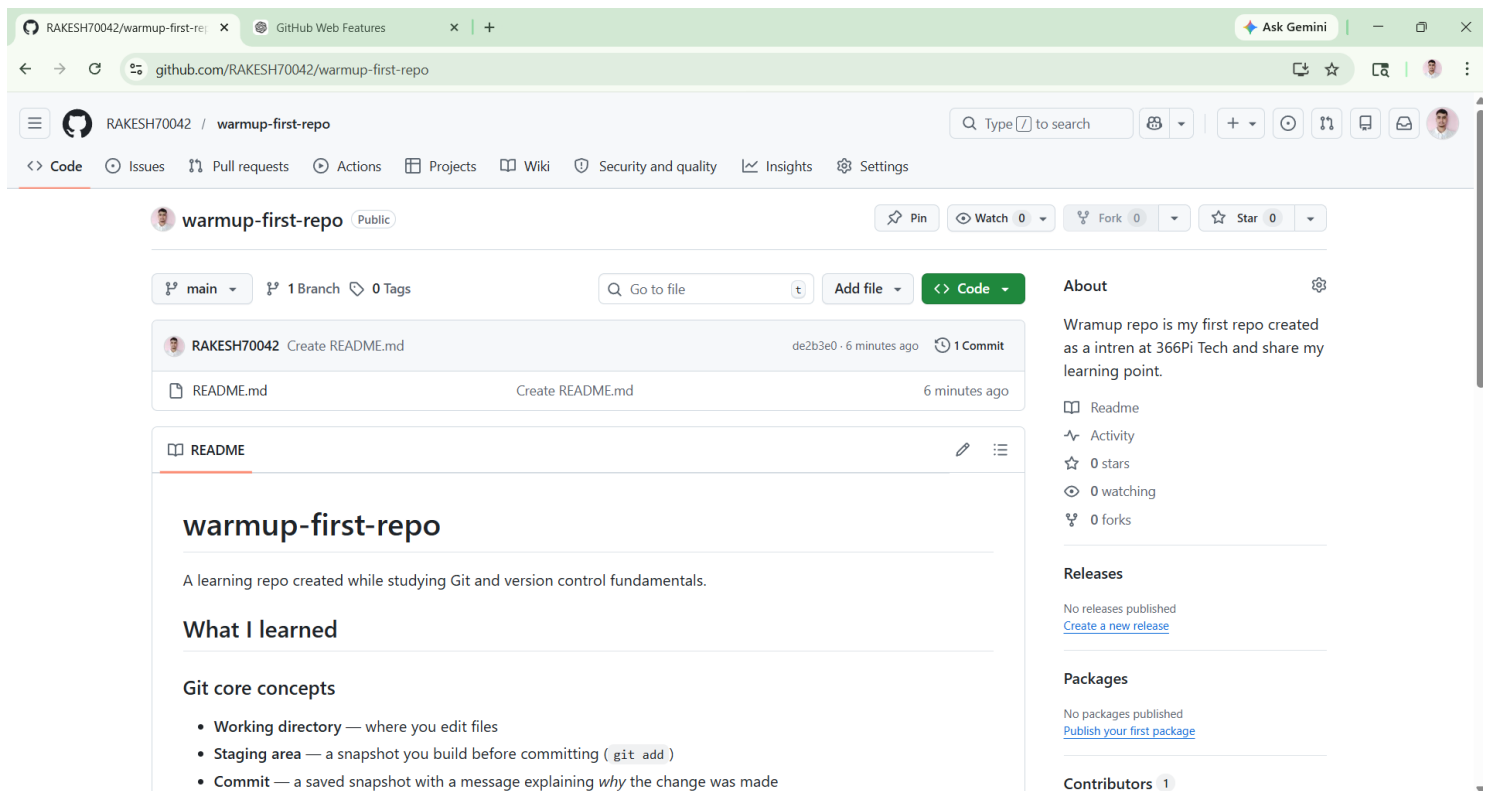
Every dev has the full history. No single point of failure.

Review gateway

Pull Requests

No code reaches main without review. PRs are built on branches.

Above Figure uses purpose only to represent the theory in creative way.



Step to Create a GitHub Repo

- I have Create a public repo named warmup-first-repo
- Push a README.md file with summary of what you learned so far
- Practicing making 1-2 commits

All highlighted topics were covered during interactions with senior guidance in the office.

3. Release Management Process

- Read internal documentation (or walkthrough with mentor)
- Understand what a Release means at FloCard
- Note how QA, Testing, FOU, and Deployment is coordinated

4. Setup Essential Dev Tools (New) Install and configure the following tools:

- Git CLI or GitHub Desktop - VS Code or Android Studio (based on project alignment)
- Postman / Thunder Client - Node.js and NPM
- Docker Desktop (basic config) Validate installations with
- version checks

5. Final Deliverables:

- A shared GitHub repo with warmup learnings
- Shared FloCard + connected with team
- One-pager write-up on products + flows
- Use case notes - Tools installed and tested